



UNIVERSITY
of VIRGINIA

SCHOOL of ENGINEERING
& APPLIED SCIENCE

Using Machine Learning Techniques to Assist Parasitic Egg Counting

EGGS BY THE DOZEN

May 3, 2024

Fede Bandoni, Haroon Zahid,
Sebastain Baldeon, Alex van
Weezendonk, Colin Chandel

Susan Skalak

Report Title:

Using Machine Learning Techniques to Assist Parasitic Egg Counting

Project:

Eggs by the Dozen

Prepared for:

University of Virginia School of Engineering and Applied Sciences

Client:

Susan Skalak

By:

Federico Bandoni

Haroon Zahid

Sebastian Baldeon

Alex van Weezendonk

Colin Chandel

First Year Engineering Center

Professor Benjamin Goldschneider

013-STO

Group 1

University of Virginia

School of Engineering and Applied Sciences

Thornton Hall

351 McCormick Road

Charlottesville, VA 22904

Contents

Executive Summary.....	1
1 Introduction.....	2
1.1 Client Problem	2
1.2 Objective and Approach.....	2
1.3 Report Organization	2
2 Problem Definition and Background.....	3
2.1 Client Interview Summary	3
2.1.1 Customer Discovery	3
2.2 Literature Review	4
2.2.1 Introduction.....	4
2.2.2 Parasites in Animals	4
2.2.3 Machine Learning Methods.....	5
2.2.4 Data Sets.....	6
2.2.5 Application Building	6
2.2.6 Previous Work.....	6
2.2.7 Gaps	7
2.2.8 Closing	7
2.3 Problem Definition	7
3 Initial Solution Ideation	9
3.1 Development of Alternative Designs	9
3.2 Selection of Initial Design.....	11
3.3 Detailed description of design	12
3.4 Work Plan	14
4 Testing Results, Analysis and Design Revisions	16
4.1 Initial fabrication and testing plan.....	16
4.2 Testing results and analysis.....	17
4.3 Informed design revision.....	17
5 Summary of final solution	19
5.1 Description of Final Design	19
5.2 Reflection of Final Solution Fulfilling Customer Needs.....	20
6 Recommendations and Future Work	22
6.1 Project Limitations and Potential Improvements.....	22
7 References	23

List of Figures

Figure 1. Alternative Solutions 11

Figure 2. Initial Design Return Options 11

Figure 3. Solution Process Flowchart 14

Figure 4. Prototype Development Gantt Chart 15

Figure 5. CNN Convolutional Layer Diagram 20

List of Acronyms

CNN	Convolutional Neural Network
FAMNet	Few-Shot Adaptation & Matching Network
FEC	Fecal Egg Count
NN	Neural Network
YOLO	You Only Look Once

EXECUTIVE SUMMARY

Small ruminants (goats and sheep) face the persistent threat of lethal parasite infections, and it is expensive and time consuming to test them for infection. These infections are dangerous, and infections caused by strongyle nematodes can be highly contagious and lethal if not detected soon enough. Because of the nature of these infections, it is highly important to detect them early before they cause a severe problem in an individual's overall population of ruminants.

These infections can be detected by completing fecal sample analysis via parasite egg counting, which allows the infection state of the animal to be determined, as well as how far the infection has progressed, and if deworming pharmaceuticals need to be applied. The various research into existing methods found a lack of effective solutions for this issue beyond the existing form of manual microscopic evaluation.

To allow farmers to test their animals more easily for infection, we attempted to create a neural network that would count the number of eggs in microscopic images of fecal matter on a McMaster slide. However, we were only able to find a handful of useful images for training our model, far short of the hundreds of thousands of images required to train standard image recognition models, making it unlikely that our model would be reliable.

In training, the model displayed an accuracy which was over 90% for most training steps. However, the validation accuracy of the model was far lower, at 55%, meaning that although the model was able to partially complete the task presented, it was not nearly effective enough to be implemented in an effective and client-satisfactory solution. As the model did not provide the accuracy needed, various methods were used to determine what the cause of error was, before being attributed to the overall lack of training data available for establishing an effective CNN for the designated issue.

Additionally, our project development was limited in a variety of aspects, including the prototyping time available, access to sufficient training data, and an overall exhaustive scope that could not be fully satisfied with the limitations imposed. Due to these limitations, the project solution was elements of a full solution rather than a collective, functional system. These project components included a CNN image recognition model and a website for hosting the model.

Ultimately, with additional data collection, physical prototyping, and dedicated hardware for model hosting, the work completed in this project would lay a foundation for an efficient and effective egg counting solution which could be implemented effectively to reduce costs and simplify the process of completing fecal egg analysis. This solution would appeal heavily to small farmers if fully implemented and would fulfill the criteria defined in this project.

1 INTRODUCTION

1.1 Client Problem

Our client's animals face a continual risk of being infected with parasites. Left untreated, these infections are lethal, and dewormer is our client's main treatment for infected livestock. However, deworming is highly destructive to the health of the animals, so deworming animals without knowing if they are infected is a very poor strategy. It's difficult to determine whether a goat is infected before their health is severely compromised by the infection, and therefore, an effective diagnostic method is necessary. Currently, veterinarians use a McMaster slide and microscope to count and identify the parasites. Our client currently pays \$16 per fecal sample per animal, and they have multiple animals. Preferably, animals should get tested multiple times a year to keep track of their health. The current process of testing animals for parasites is tedious because it is time-consuming to have to manually count all the eggs and because it requires the services of a veterinary technician. Additionally, testing is costly, considering the cost per sample adds up rapidly.

1.2 Objective and Approach

The objective of this project was to develop a low-cost method for Strongyle Nematode egg identification in small ruminant fecal samples. This problem was approached from a machine learning perspective, with the main function revolving around neural network image identification of strongyle ova (eggs) in solution. This would mitigate the problem by allowing farmers with microscopes to quickly determine whether their cows are infected with parasites, but does not completely solve it, since farmers would need their own means of obtaining microscopic images of fecal samples.

1.3 Report Organization

Chapter 2 describes what we learned about the problem through background research and interacting with our client. **Chapter 3** details our approach to solving the presented problem, as well as the reasoning behind the final decision. **Chapter 4** details the design and testing process of the final prototype, as well as the redesign process. **Chapter 5** details the final solution components, solution efficacy, and client response. **Chapter 6** details the limitations and potential aspects for improvement of the current design, as well as future work for the project. **Chapter 7** includes the references for the project.

2 PROBLEM DEFINITION AND BACKGROUND

To generate an effective and successful solution to the problem, an in-depth examination of various components of the problem being addressed was essential. Firstly, preliminary research and interview directly with the client allowed definition of some of the more particular criteria for the solution, while also generating a variety of concepts for potential solutions, as well as a general thought process for the overall project. By examining existing solutions to this issue and their current flaws, a unique solution which is intended to both solve the problem and avoid such flaws was developed.

Given the problem, our solution must create a way for our client to accurately tell whether each of their animals has a parasite infection. According to our client, our solution must fit the following additional constraints: it must not require hiring a technician; it must cost less than \$16 per test and require less than 4 hours per test. Our client's current process of finding parasites costs \$16 per sample and requires roughly 4 hours of driving to and from the vet, so any useful solution must be less costly and inconvenient for our client. Hiring a technician would also be too expensive to be practical for our client. Based on information from our client, we've determined criteria to attempt to fulfill criteria as well as we can. Our client and most farmers would be reluctant to buy a microscope, so our solution would be much more effective if it did not require a microscope. In parasite detection, false negatives would likely lead to animals subsequently dying of parasite infections, and false positives would result in dewormer being used on animals prematurely, which severely disrupts their digestive systems as well as builds the parasite's resistance to dewormers over time. Consequently, our solution should minimize false negatives and false positives as much as possible. It would also be useful for our client to be able to input additional health data to inform the detection of parasites.

2.1 Client Interview Summary

The client interview started with the client introducing themselves, their name is Susan Skalak and they're a former UVA professor and a small-scale farmer. They then presented a slideshow with some background information that they thought would be useful for the teams, for example, some criteria and constraints. Once the slideshow had concluded, the client was open to answering questions from all the teams present. Teams took turns asking questions and added follow-up questions when necessary. There were many significant takeaways from the interview. One of them was that the client was very adamant about not wanting to use a microscope. Another takeaway was the detrimental effect of overcounting eggs. If the app were to overcount the number of parasitic eggs present in an animal's body, then that would cause the client to give the animal de-wormer, which is not something that should be given unless it is necessary since it's very harsh on their body. Additionally, we were able to determine the operating system that our client utilizes, which is crucial information to have when choosing the kind of application or website to develop so that it's compatible with their device.

2.1.1 Customer Discovery

Identifying our client was one of our team's initial research projects. It was discovered that our client was a UVA professor and that she majored in mechanical engineering. This was important because it meant that she'd be able to understand us when we used some technical language in our interview questions. After reading our customer's problem definition we immediately recognized that any possible solution would probably involve machine learning. Therefore, our group investigated machine learning algorithms that are currently in use and that could potentially be applicable to our issue. To fully comprehend the nature of the issue and the experiences our client faces, we also conducted some research on the procedure for collecting fecal samples. We also had to find some existing parasitic egg datasets since our client did not have any available to give to the teams.

Typically, the customer and user will be the same person since whoever purchases our product will have to use our app or website to access the product. In our case, the client, customer, and user are all the same person: a farmer with many animals who are affected by parasitic eggs. Therefore, our client will also be a user of our fabricated solution, whether it be a website or app.

2.2 Literature Review

2.2.1 Introduction

Parasites in farmyard animals provide a significant source of concern for farmers, especially due to the increasing resistance to treatments that parasites have begun to develop. Traditional methods for parasite detection are often expensive and unsustainable for small farmers, and without widespread development of modern, cheaper methods, there is an urgent need for improvements in parasite detection. This literature review serves to display the potential implications of utilizing a conventional neural network (CNN) or Few-Shot Adaptation & Matching Network (FAMNet) to develop a more cost-effective solution for identifying parasite eggs in fecal samples, therefore allowing small farmers to identify parasite concentration and provide treatment, when necessary, more effectively.

2.2.2 Parasites in Animals

Goats, sheep, horses, and other farm animals are plagued with a variety of parasites, the most infectious and deadly of which is the barber pole worm. However, various other parasites also infect these animals, all of which are generally part of the group of strongylid nematodes, which are an intestinal parasite. Because of the evolutionary nature of these parasites, use of parasitic removal drugs is becoming less and less effective, and animals cannot be protected by always medicating them due to the stress that parasite removal drugs cause to the body of the host animal. The Barber Pole Worm specifically has seen an increase and spread of resistance worldwide to all four of the primary classes of drugs used to treat infections (Schwarz *et al.*, 2013).

While full prevention of strongylid nematodes is impossible, higher quantities of parasites in a host can lead to a significant reduction in health and can eventually be fatal. Due to the dangerous nature of these parasites, fecal sample evaluation is performed on farm

animals to ensure egg concentration does not exceed safe values for the Fecal Egg Count (FEC).

Currently, FEC is measured with a McMaster slide, which involves the preparation (mixing and filtering) of a salt solution to isolate strongylid nematode eggs in the sample (Zajac *et al.*, 2014). Once isolated, eggs are placed into a McMaster slide, which is an optimized microscope slide for FEC determination. Once the slide is prepared, a microscope is used to count the eggs inside of the observational area of the slide and multiplied by a known constant to determine the FEC per gram of fecal matter. This procedure, in its current state, is impossible for small farmers to perform regularly due to the lack of materials and high cost of exporting labor to veterinary laboratories. As a result, the client suffers from having to pay this high cost for fecal sample analysis.

As of right now, using the McMaster slide and having this analysis performed in a lab is the only method used regularly for fecal sample analysis. Other developments are underway, but progress in this field is slow due to the unique nature of this issue and lack of dedicated resources for improving the quality of strongylid nematode testing. For example, a 2011 study performed on sheep in Australia tested a new molecular method for identifying strongylid nematodes, allowing an analysis of the percent composition of the various species of parasites as well as a determination of the overall number of parasites (Roeber *et al.*). While this study displayed that new methods could be potentially feasible for parasite analysis in farm animals, the high cost of developing an efficient new solution became rapidly apparent. As a result, the molecular method used in this study has not been significantly improved to be more cost-effective than the slide method detailed above.

2.2.3 Machine Learning Methods

For our client, a machine learning algorithm will be developed to detect parasitic eggs in a sample. There are two main methods in object detection algorithms: convolutional neural networks (CNN) and Few-Shot Adaptation & Matching Network (FAMNet).

According to IBM (n.d.), CNNs are distinguished by their “superior performance with image, speech, or audio signal inputs.” CNNs start with a convolutional layer which simply divides the image into smaller images known as kernels to better analyze each smaller image. Then, it begins combining the smaller images to form slightly bigger images and again analyses the images. Finally, it ends with a fully connected layer to be able to draw its conclusions. Each layer in a CNN adds complexity, and more layers can be added to analyze the images more and make more connections. Due to its complex filters and abilities to analyze images, CNNs can be used for object detection and counting. Krizhevsky *et al.* (2017) show just how complex a CNN can be made. Their model has eight total layers, five of which are convolutional layers with different sizes of kernels in each layer. Max pooling is applied to certain layers to summarize the outputs of adjacent kernels. After the convolutional layers, three fully connected layers follow that are then fed into a soft-max layer that then gives an output. With this size of CNNs it is very common to see overfitting of the data. A solution to this is to add drop-out layers which makes it so that kernels and layers cannot depend on each other.

According to Seguí *et al.* (2015), “the remarkable performance of CNN ... suggests the use of this technique to learn to count objects in a scene.” Ideally, the CNN can be fed an image and return an estimated number of items in the image. However, one of the drawbacks on CNNs is their long training times. Since each image gets broken down into smaller images, the whole network takes longer to run. However, this can be solved by training with a smaller portion of the data to best see what layers and filters can be applied, and once there is high performance with the smaller data set, the entire data set can be used.

A newer proposed solution to object counting proposed by Ranjan *et al.* (2021) is Few-Shot Adaptation & Matching Network (FAMNet). FAMNets were developed as a better counting algorithm that eases the drawbacks of CNNs and regression models. FAMNets do not distinguish between a test and train data set which allows for analysis of smaller data sets. The network can return a density map to then count the specified object. It is unclear if this method provides any performance improvements compared to a CNN, but it is good to have another method of analysis available.

2.2.4 Data Sets

Training a machine learning model requires a dataset; in this case, the data set would consist of images. Ideally, the images would also have a dataset with information about the images including potential quantities and qualities about the images. In this project, helpful data would include the actual number of parasites in the image and the species of parasites found in the image. Palasuwan *et al.* (2022) have a great dataset available that has 11 species of parasitic eggs from fecal samples with 1,000 images per species. This dataset would give us a wide variety of images and a good start on training. Ideally, a data set that includes the same species that the client handles would be better for the training. However, even if one is not found, this data set has the capability to train a model well enough to handle just egg counting but not necessarily species identification. For the purposes of this project, a network which handles general strongylid nematode ova identification without specific species identification would still provide ample information to a farmer for the purposes of animal care.

2.2.5 Application Building

Ideally, the client would like an app where she can upload images of fecal samples and have the app count the eggs within. To do that, we would have to build an app that either hosts the machine learning model or hosts the model in a cloud and have the app access that. Kalinin (2022), provides step-by-step instructions on how to develop a machine learning model and deploy it on an app.

2.2.6 Previous Work

There appear to be similar implementations of solutions to our problem, but none that fit the client's exact needs.

Kumar *et al.* (2023) used a CNN to detect intestinal parasitic eggs in human samples. Their model yielded a very high success and performed much better than previous iterations. However, it is unclear if the model could be used on animal fecal samples. Naing *et al.* (2022)

used an older version of the model used in the previous study to also identify protozoan cysts and helminthic eggs in human fecal samples.

Akintayo et al. (2018) used a Convolutional Selective Autoencoder to detect and count soybean cyst nematodes in soybean plants to assist with plant treatment. This study is different in that it does not use a CNN but still uses a convolutional deep learning model. Yet, autoencoders still use CNNs in their encoding and decoding parts.

2.2.7 Gaps

We currently lack a better data set for our project. Preferably, we would use a data set that has the same species our client deals with, but there does not seem to be any.

Additionally, the gaps in biological research involving strongylid nematodes limit the potential options in which we could biologically improve identification of the sample, such as staining eggs with an identifying solution. Because of this, the nature of our solution needs to be involved with identifying eggs in unmodified, conventional McMaster slides.

2.2.8 Closing

Ultimately, although our research provided a variety of helpful foundations for the creation of our own unique solution, it also displayed the current void in small-scale implementable solutions for small farmers. Because of this project's nature, components of previous solutions along with our own uniquely designed aspects offer us a distinct potential for implementing a low-cost, small-scale, and effective solution.

2.3 Problem Definition

In the agricultural domain, livestock including goats, horses, and sheep suffer from the affliction of deadly parasites. Small scale-farmers and caretakers of animals face significant challenges in detecting parasites efficiently and affordably. Conventional methods of detecting parasites are not only time consuming but also expensive and often require the expertise of veterinary professionals that many farmers cannot access with ease. As a result, small-scale farmers who usually run on low budgets and limited resources, cannot afford to test for parasites at the ideal frequency, often risking the deaths of farm animals or risking using dewormer more often than necessary. Compounding this issue, parasites are evolving to become more resistant to dewormers, and dewormers are destructive to farm animals' digestive systems, making using unnecessary dewormer a highly unsuitable solution. On another note, the lack of accessible and affordable diagnostic tools intensifies the already formidable task of managing farm animal health, leading to increased financial strain and diminished productivity. Therefore, implementation of a cheaper and faster method of identifying parasitic infections among livestock for farmers would be extremely important as it would help them to overcome the hardships being faced due to the shortcomings of the current diagnostic process.

Given the problem, our solution must create a way for our client to accurately tell whether each of their animals has a parasite infection. According to our client, our solution must fit the following additional constraints: it must not require hiring a technician; it must

2 PROBLEM DEFINITION AND BACKGROUND

cost less than \$16 per test and require less than four hours per test. Our client's current process of finding parasites costs \$16 per sample and requires roughly four hours of driving to and from the vet, so any useful solution must be less costly and inconvenient for our client. Hiring a technician would also be too expensive to be practical for our client. Based on information from our client, we have determined criteria to attempt to fulfill criteria as well as we can. Our client and most farmers would be reluctant to buy a microscope, so our solution would be much more effective if it did not require a microscope. In parasite detection, false negatives would likely lead to animals subsequently dying of parasite infections, and false positives would result in dewormer being used on animals prematurely, which severely disrupts their digestive systems as well as builds the parasite's resistance to dewormers over time. Consequently, our solution should minimize false negatives and false positives as much as possible. It would also be useful for our client to be able to input additional health data to inform the detection of parasites.

3 INITIAL SOLUTION IDEATION

3.1 Development of Alternative Designs

The problem description initially prompted two main ideas for solution development. The first of these ideas was to implement an image-recognition machine learning model that would return a fecal egg count after performing an analysis of uploaded sample images. The secondary idea was to perform a biological analysis of the egg count through chemical mechanisms to allow eggs to be more easily counted, therefore relieving the need to train a machine learning model to address the problem. However, a proper implementation of using a biological mechanism would still require expensive chemical processes as well as specialized equipment such as a cell counter, which is therefore not feasible compared to a machine-learning solution.

It was initially anticipated that the customer would possess a microscope to capture images of their slides and submit them via the app or website. But after speaking with our client, it was determined that the client wished not to use a microscope, as they did not own one. Because parasite eggs are incredibly small and usually require at least 100x magnification for viewing, the project materials that could be obtained in a limited timeline would not be able to deliver high-quality photographs of parasite ova without the use of microscopes. However, we were able to learn from another team during the client interview that there is an existing product that works as an attachment for a mobile device and functions as a microscopic lens.

Due to the nature of the problem, an image-recognition model was the primary mechanism around which solutions would function. As a result, the dynamic section of producing ideas revolved around the methods through which the client would interact with the model, such as the upload of data or how the egg count would be returned. As a result, the three primary ideas generated revolved almost entirely around the same machine learning model structure, with the return differing slightly between each idea. Since our final solution incorporates features from each of the initial solutions, we feel that we have given each prospective proposal the same amount of thought.

The first solution, demonstrated in Figure 1 below, was developed as an ideal solution to the problem, with a program flow that involves the user directly uploading data, processing the images with a neural network, and returning an exact egg count while saving data for record-keeping. Because this solution would return an exact value for the FEC, it would be the least labor-intensive for the user. However, the feasibility of this solution is significantly lower than that of the other potential solutions, because the chance of developing an image recognition network with this degree of accuracy given the limited timeframe is less likely compared to the other potential solutions.

The second solution, demonstrated in Figure 1 below, we came up with was called the "Plus or Minus Model." This model revolved around many of the same fundamental concepts as the first one but would return an estimated value with a delta value for accuracy. What's considerable about this model is the fact that it's much more achievable than the first

3 INITIAL SOLUTION IDEATION

solution. However, the fact that it provides a delta value of accuracy means that the model will overestimate or underestimate the true number of eggs. Ideally, the outputs would be saved to the app/website for record-keeping.

The third solution, demonstrated in Figure 1 below, was dubbed the “Assisted Hand Counting Model.” This model was established as the most realistic functional design, returning information that the user would process manually by selecting if potential image matches were eggs or not. An advantage of this model is that it’d have higher accuracy since the user would be confirming the possible matches. Since this model only assists the user, that’d mean that the user would ultimately be the one counting eggs, which could be time-consuming and tiresome to do.

The exact method of each of these return methods was created by various members of the group, with different group members developing the ideas for each of the three return possibilities. Figure 2 below demonstrates possible options that the program might return once data has been processed, with each of the return options described in more detail below.



Figure 1. Alternative Solutions

Method (1)

Returns exact egg count e.g.
"FEC is 900/g. Recommend dewormer for lactating does"

Method (2)

Returns potential egg count e.g.
"FEC is 650– 800/g. Recommend reevaluation"

Method (3)



Figure 2. Initial Design Return Options

3.2 Selection of Initial Design

All three of our initial ideas require a machine learning model that can identify eggs; however, we needed to know how accurate our model would be before committing to a

particular idea. The logical choice of action is to just build the model and then decide whether to output a range of values, a single number, or a collection of photos. To put it briefly, we are putting off settling on a single design while maybe incorporating elements from many. Naturally, the perfect model was our top choice, but because it was so perfect, it was out of scope for this project and not very practical. The roundtable discussion was beneficial because it allowed us to practice presenting our work, however we did not receive any helpful feedback.

The number of eggs the model identifies will be output if it counts them with 100% accuracy. We'd ask the model to output the maximum and minimum number of parasite eggs expected to be present in the sample, assuming it is mostly accurate. We will have the model indicate the images it labels as possibly having an egg for the client to sort if its accuracy is moderate, or at least useful.

We would've preferred to have chosen the perfect model; however, we felt that it was not reasonable to complete it within the timeframe that we were given. Additionally, we all agreed that we needed a machine learning model and that the precise format of our output would depend on the model's accuracy, so there was no need for us to vote or use a decision matrix.

3.3 Detailed description of design

Our team has implemented a solution which involves making a machine learning algorithm and a website/app to host and utilize the algorithm. Upon accessing the website/app, clients will need to upload images of the egg slides they have prepared for inspection. These images will then be processed by the algorithm.

Our algorithm operates in a bilateral approach. Initially, it analyzes the images, identifies eggs that are infected. In this case, the algorithm will correctly provide the user with precise results, detailing the number of infected eggs detected.

However, given the complexity of parasite detection and the accuracy of the model based on the training, we incorporated a second mechanism in our algorithm for cases where the algorithm is unable to identify all the eggs with zero uncertainty. In such cases, instead of delivering inconclusive results, the algorithm will provide the user with images of eggs that could not be detected by the algorithm. And through an interactive interface, the users are asked to inspect the images manually and select either 'Yes' or 'No'. Doing this will ensure that the algorithm is not providing any false results. This final output is the combination of the three solutions we came up with and it's the most ideal to ensure consistent accuracy.

Furthermore, all the results and data will be stored in a server the website/app is connected to. It will allow clients to look at their history and confirm anything they had forgotten to do.

Our solution is designed with a long-term perspective, a sustained life cycle supported by regular maintenance and updates. The only physical component to our solution is a

microscope to capture images which will need to be bought again if damaged. We expect the algorithm to remain relevant for several years, with adjustments from time to time. Maintenance tasks usually will include refining the machine learning algorithm to ensure its efficiency and updating the app/website to prevent any security threats and enhance user experience, and a small monthly fee for website maintenance which might not be necessary. With it, there will not be any need for a complete reconstruction from scratch rather it will only need improvements built upon existing infrastructure.

Risks to our product include the consistent accuracy of the algorithm and security concerns related to data privacy as everything is accessed online. The algorithm potentially can act on inaccurate information which can cause some error in the output, leading to unnecessary deworming or goats dying from parasite infections.

3 INITIAL SOLUTION IDEATION

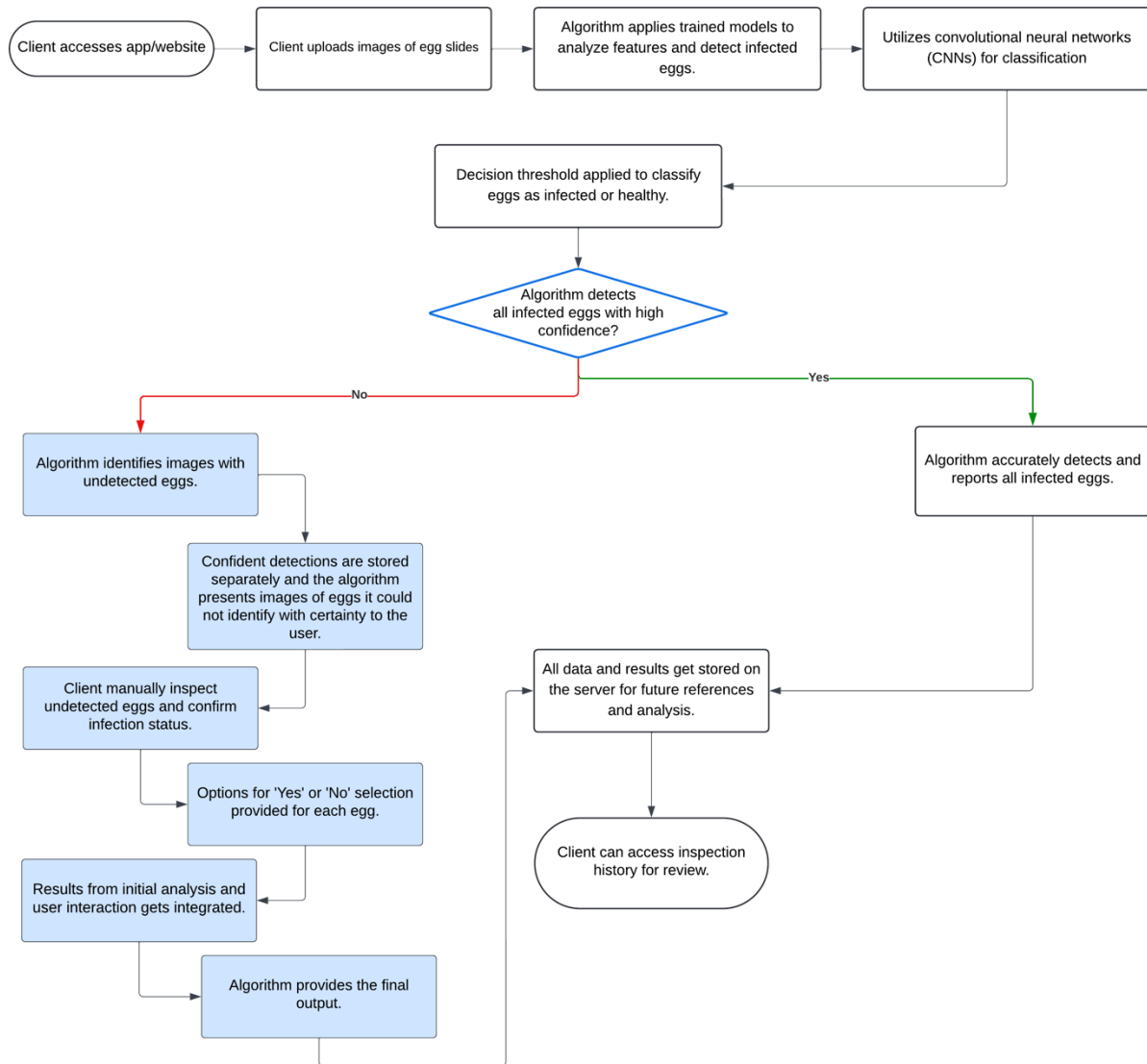


Figure 3. Solution Process Flowchart

3.4 Work Plan

To ensure development proceeds at a rapid and efficient pace, the chart seen below in Figure 4. will be used to ensure that all subdivisions of the project are completed in an efficient and orderly manner.

3 INITIAL SOLUTION IDEATION

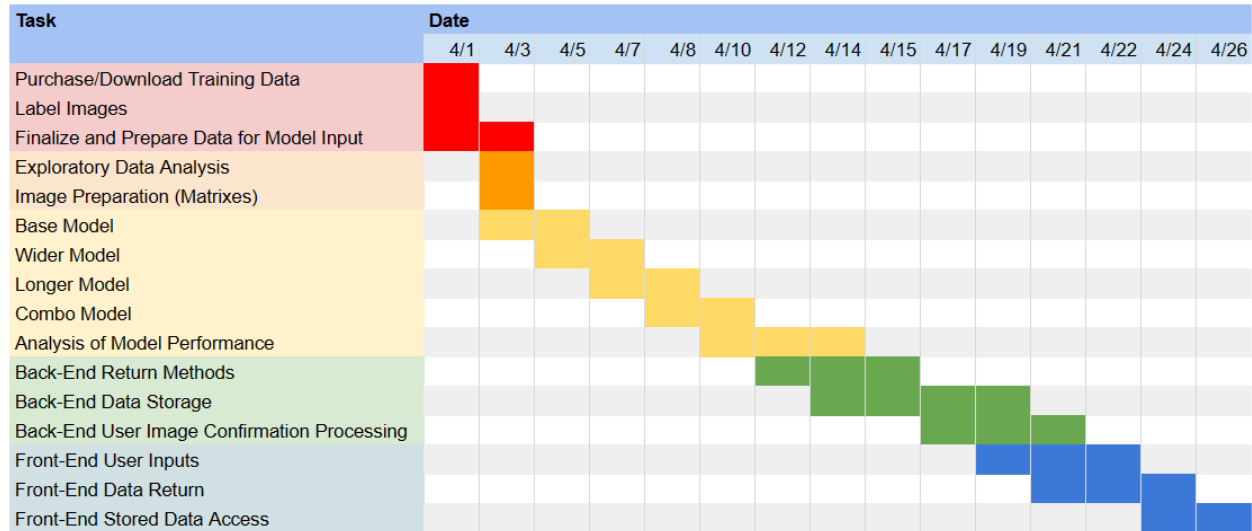


Figure 4. Prototype Development Gantt Chart

For this project's purposes, no physical materials were required. However, training data requirements may necessitate the need for purchase of a training database or other related information necessary for training an image recognition model in the future development of the prototype solution.

4 TESTING RESULTS, ANALYSIS AND DESIGN REVISIONS

In this chapter of our project, we started the fabrication of our model by first gathering as many images as possible to use as data to train the neural network of our model. We then simultaneously began the software development aspect of our model and the website that would eventually have the model imported. Once we had finished the basic fabrication of our solution, we then began testing according to our testing plan, which is gone into more depth later in this chapter. The results recorded from testing the model gave valuable information, which then led to changes in our software with the process of iteration.

4.1 Initial fabrication and testing plan

Our fabrication process began with looking for potential data sets of images to use for our model training and validation. Our object was to get parasitic eggs that were as close to a match to the species of eggs that our client handles for better accuracy and real-world application. Unfortunately, we discovered that there were not many images pertaining to what we were looking for. We wanted images that had lots of eggs on one slide so that the model could learn to detect and count all visible eggs in an image. However, we would find many datasets that instead of counting eggs, focused on classification, as in classifying what species of parasitic egg was present and so the images were of single eggs. Realizing this we created our own data set using a few images we were able to find and modifying them to get more images.

These images were obtained through directly contacting a large range of veterinary offices, veterinary diagnostic laboratories, and veterinary departments associated with major universities. However, even with the large number of veterinary labs contacted, the number of images collected for training the model was very small and predicted to be inadequate for training the model. Of the laboratories contacted, only the veterinary diagnostic lab of UC Davis replied, offering training data that was divided into about 40 images for training the model.

Our data set consisted of about 40 JPEG images along with a CSV file that includes the image file names and how many eggs are in each image. Using the image and files a model was started. The images were separated into a 70-30 split, where 70 percent of the images were used for training and 30 percent were used for testing. Of the 70 percent, 30 percent was used for validation of the training. This is a common technique used to ensure the model is trained properly without showing the model the test images.

The model we used was a Convolutional Neural Network (CNN). CNNs are the industry standard for machine learning image analysis. They are an advanced form of Neural Networks that use convolutional layers which allow the model to see not only the indexed data but also the data around it allowing the model to a "bigger picture." One of the massive advantages of utilizing a CNN is that they can determine intricate features in a localized region, capturing fine details as well the "bigger picture" mentioned previously. This allows CNNs to process spatial information efficiently and effectively, which makes recognition across a diverse dataset possible. Because of this adaptability, a CNN is the ideal model for creating the

desired approach for this project, as the project itself revolves heavily around big-picture pattern recognition.

Error analysis of a CNN model is usually performed through two primary methods, accuracy calculations and visual inspection. The CNN developed for this project was tested using these two methods. The model's accuracy is determined in the model's training sequence, in which the number of correct matches over the number of total matches provides a mathematical percentage representing how often the model predicts a match correctly. This accuracy value is the primary value in understanding model function; specifically, the CNN accuracy value can be used to determine how often the model will be correct/incorrect, allowing the project to be further developed with an appropriate return method depending on the model efficacy. Additionally, analysis can also be performed via a qualitative observation method, in which the model's responses given various specific inputs are used to determine potential sources of error in the model. This method of analysis is performed to determine any possible error cases that cause the model to incorrectly identify a match or fail to identify a match. Combining these two error analysis methods allows the developer to determine the model efficacy and its consistency. As a result, the data used for testing determines the model's applicability to the defined problem.

4.2 Testing results and analysis

In the training environment, our model showed an accuracy of 55%. Because of the low accuracy of the model overall, it would be relatively ineffective in identifying eggs in a sample. Although the exact cause of the general failure of the model could not be determined, it was speculated that the set of image data input into the model to train for egg identification was inadequate in both the quantity of images available as well as the diversity between the images utilized.

This lack of data quantity led the model to overfit to the specific input data, meaning the model was essentially guessing a value for each of the input images. This is a result of the specific size of the training set, which included 40 total images, rather than the tens of thousands needed to train a CNN. Although this result was certainly expected as a possibility, the exact extent to which this lack of image diversity would affect the model was not known until the training had occurred.

This effect is due to the nature of training CNN models. These models are optimized for training containing a vast and diverse input dataset, meaning that the limited quantity of the training data utilized is suboptimal. Although a CNN is the optimal model for this project, the massive data limitations resulting from a gross lack of training material ensured that the model is neither consistent nor effective.

4.3 Informed design revision

Model training began with a typically basic CNN, with a lower number of layers and kernel sizes. Typically, a base model is used to see how a model interacts with the data and often performs minimally. As expected, validation and testing accuracy was low, but what was more concerning was the training accuracy. High training accuracy with low testing and

validation accuracy usually indicates overfitting of the model. This is to be expected because of the lack of diverse and expansive data. So, that model was then retrained with a lower batch size, but much more epochs. Lowering the batch size lowers the number of images that are fed into the model with each run. An increased number of epochs allows the model to run more batches and see more images for a longer time. As expected, the model performed a bit better, so that model was followed up with a wider model which means larger layers and larger kernels. This model performed better, so instead of 10 epochs, we ran 50 epochs to see how the model would run in the long term. This model ended up being our highest performing model. However, to ensure that we looked at possibilities, a much narrower model was made to see if overfitting could be resolved. Once the model ran, we saw a significant drop in performance and decided to not continue with that type of model.

To access the model, we made a website. To make the website, we chose Replit to be the platform. The website initially consisted of HTML and CSS to provide an interface for the user to interact with. It includes information about various types of parasites and techniques to prevent them. The website then includes another page, where the ML model is supposed to be embedded and a space for users to upload the pictures to be processed by the model. Throughout the entire process, we encountered general coding errors. To be specific, the first obstacle was the image uploading process. The JavaScript code responsible for handling file uploads had errors in it and was not letting the user access the files on their computer. Later, the main problem comes with accessing the ML model, coded in python and presents compatibility issues with the web environment. Integrating python-based machine learning models into websites requires using additional frameworks such as Flask. We worked on it, but it did not end in success. We were encountering errors we did not know how to resolve. Without the ability to integrate ML models effectively, the functionality of the website remains limited. Given that, we chose not to host the website as that would serve no purpose.

Moving forward, addressing these challenges will require a reevaluation of the website and ML model architecture and development process. It will require more time and specific hardware for proper functionality and scalability of this project. Big models like these are not supposed to be deployed on standard hardware configurations and require specialized infrastructure.

5 SUMMARY OF FINAL SOLUTION

5.1 Description of Final Design

The final solution developed consisted of two primary elements: the trained CNN and the website developed for model hosting. Although the developed model was not fully implemented onto the website due to hardware limitations, the website was designed around the capacity to insert model inputs, and therefore, with a dedicated server to host the model, would function as intended.

Although many CNN models were tested, the model with the highest performance was the model with the narrowest and least number of layers. This is likely because of the lack of images and necessary data to make complex layers and decisions when training the model ensured that a simple model which essentially guessed values would be more accurate.

The final iteration of the model had many changes from the solution detailed during preparation due to three primary causes: (1) Data limitations (2) development time constraints and (3) hardware limitations.

(1) The primary cause for the inefficacy of the final model was due to the limitations of data provided for training, which causes the model to not effectively utilize its own learning capacity during the training phase. Due to the lack of numerous and diverse training data for the model, the full effectiveness of the convolutional layer utilization was not realized.

(2) Additionally, the development time for the project overall heavily limited the capabilities of the team to acquire the necessary data for training a model of this variety. However, because the project's groundwork had already been established, implementation of an alternative model was impossible within the time constraint.

(3) In order to implement the model onto a website, dedicated hardware is required to host the model and its weights. Since our model did not have optimal performance, it was decided to avoid hosting the model as that would accrue unnecessary costs.

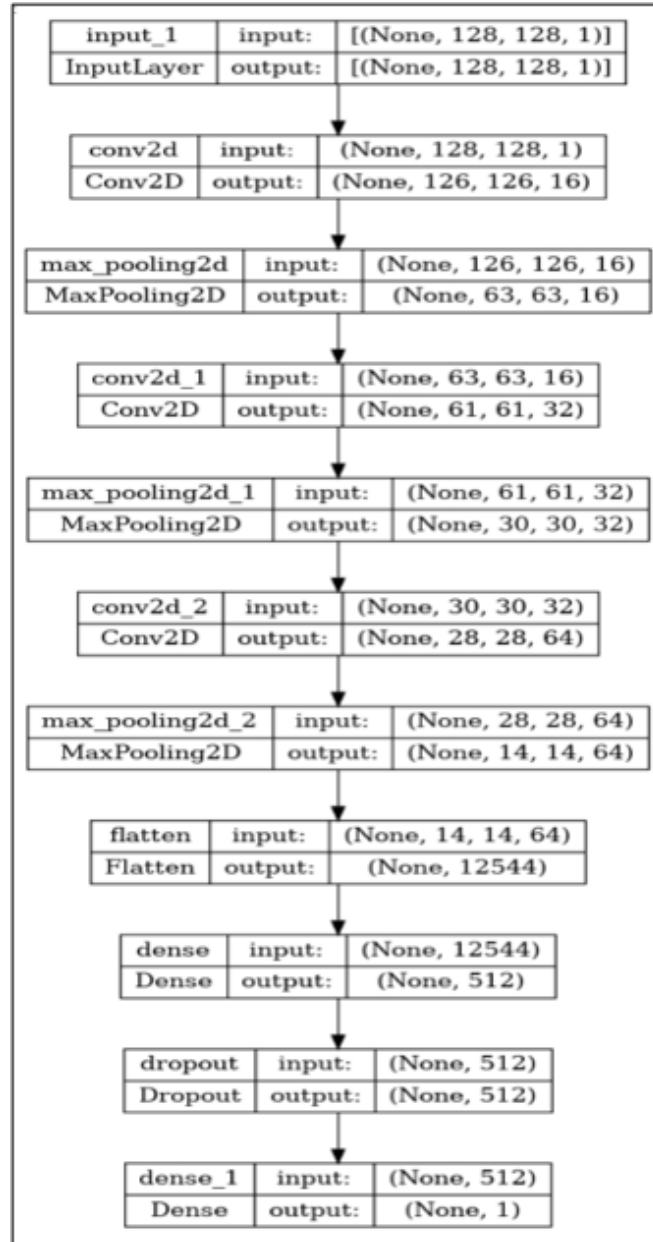


Figure 5. CNN Convolutional Layer Diagram

5.2 Reflection of Final Solution Fulfilling Customer Needs

Our design meets all of the initially designed constraints, since it does not cost money to use, does not require the services of a technician, and does not take a long time to use. However, we did not meet our own criteria of eliminating false positives and false negatives, and notably, it is not possible for the client to access our solution.

While the testing accuracy was above 50%, the model is not reliable enough to use to make decisions about whether an animal is infected with parasites. This is mainly due to inadequate training data and the low margin of error for parasite diagnoses via McMaster

slides. Additionally, there was a possibility of making a model that only detects if there are enough eggs to require treatment, but it was decided that that would not provide enough information to the client and would not allow the client to know if the animal was improving.

With so little training data, the model has no experience with images with numbers of eggs outside of a few small ranges, images that include things other than eggs, and images with different arrangements of eggs from the few in our dataset. This means the model might struggle to confirm when there's only 2 eggs in an image or if there's more than 20, since those are outside the range of eggs in our dataset. Additionally, we do not know if the model will be confused by various non-egg shapes or if it will correctly not count those as eggs. Furthermore, the number of eggs in a fecal sample is estimated by multiplying the number of eggs in an image by a large factor, such as 50, which means the model miscounting by a few eggs can easily lead to a misdiagnosis or the wrong treatment plan for the animal in question.

Finally, it was determined that the project solution initially generated in the ideation phase was significantly beyond the scope of the limited time and resources available. As a result, the final solution developed is a component of an ideal solution, and not an exact solution to the client problem itself. As a result, the components of the solution developed for the project are a step towards a complete solution to the client issue, however, they do not fully encompass all aspects of a complete solution to the initially proposed problem.

6 RECOMMENDATIONS AND FUTURE WORK

6.1 Project Limitations and Potential Improvements

One of the primary project limitations was the lack of training data available for training the CNN. Because the greatest strength of a CNN is the convolutional layers that allow the model to observe a region around the specific point of observation, the lack of diverse images for training the model severely impacts the efficacy overall. To properly train a CNN for strongylid nematode ova detection, a training dataset containing thousands, tens of thousands, or even hundreds of thousands of images would be necessary.

To improve this data collection, it would be critical to acquire thousands of labeled images of McMaster slides of fecal samples, with large varieties in egg count, egg arrangement, and arrangement of non-egg objects, requiring far more communication with various veterinary diagnostic laboratories. This increase in the data collection would also require writing JSON files which label eggs in individual images, which would need to be at least semi-automated for feasibility. Overall, these improvements in data collection and processing would greatly improve the efficiency of the model, however, the improvements to data collection and processing would add a large additional development time which was not available within the limitations of the current project.

Additionally, to improve upon the product, switching to a different machine learning model may be beneficial. CNNs are very powerful when running classification datasets but cannot make their own classes. What this means is that if the model has never seen an image with 49 eggs, but then is shown an image of 49 eggs, it will not be able to correctly classify the image. Therefore, switching to a You-Only-Look-Once (YOLO) model may be required for further improvement of the project. A YOLO model is another type of neural network, but it uses images, bounding boxes, and probabilities to make its guesses. The bounding boxes are sets of points that outline the objects to be detected and counted and assist the model in the training sets. The model then outputs points on the test set. This model is very powerful as it can make its own classes and does not have to see an example of every possible scenario to be able to guess it later. However, since this model requires bounding boxes, an additional JSON file must accompany the images to hold all the required information for the model. This alone will take some time on top of the time required to collect the images themselves.

After all of data is collected, compiled, and the model is trained, the model and its weights would have to be hosted somewhere to access them from the website. The model could either be hosted by a cloud client where they have the offsite hardware that we would access through a wireless connection, or the model can be hosted locally on dedicated hardware. This would allow the model to be directly accessed by a user, allowing the user to take images, submit them into the system, and then be provided a return value of the fecal egg count.

7 REFERENCES

- Akintayo, A., Tylka, G. L., Singh, A. K., Ganapathysubramanian, B., Singh, A., & Sarkar, S. (2018). A deep learning framework to discern and count microscopic nematode eggs. *Scientific Reports*, 8, 9145. <https://doi.org/10.1038/s41598-018-27272-w>
- Bandoni, F. (2024). Analyzing NIH Chest X-Rays using CNN models. *Kaggle*.
<https://www.kaggle.com/code/federicobandoni/senior-capstone-project>
- Kalinin, K. (2022, March 23). Machine Learning Mobile App Development: All the Whys and Hows. *Topflight*. <https://topflightapps.com/ideas/how-to-create-a-machine-learning-app/>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2017). *ImageNet Classification with Deep Convolutional Neural Networks*. University of Toronto.
https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf
- Kumar, S., Arif, T., Ahamad, G., Chaudhary, A. A., Khan, S., & Ali, M. A. M. (2023). An efficient and effective framework for intestinal parasite egg detection using yolov5. *Diagnostics*, 13(18), 2978. <https://doi.org/10.3390/diagnostics13182978>
- Naing, K. M., Boonsang, S., Chuwongin, S., Kittichai, V., Tongloy, T., Prommongkol, S., Dekumyoy, P., & Watthanakulpanich, D. (2022). Automatic recognition of parasitic products in stool examination using object detection approach. *PeerJ Computer Science*, 8, e1065. <https://doi.org/10.7717/peerj-cs.1065>
- Palasuwan, D., Naruenatthanaset, K., Kobchaisawat, T., Chalidabhongse, T. H., Nunthanasup, N., Boonpeng, K., & Anantrasirichai, N. (2022). *Parasitic egg detection and classification in microscopic images* [dataset]. IEEE. <https://ieee-dataport.org/competitions/parasitic-egg-detection-and-classification-microscopic-images>
- Ranjan, V., Sharma, U., Nguyen, T., & Hoai, M. (2021). *Learning to count everything*. Stony Brook University.
https://www3.cs.stonybrook.edu/~minhhoai/papers/fewshot_counting_CVPR21.pdf

- Roeber, F., Jex, A. R., Campbell, A. J. D., Campbell, B. E., Anderson, G. A., & Gasser, R. B. (2011). Evaluation and application of a molecular method to assess the composition of strongylid nematode populations in sheep with naturally acquired infections. *Infection, Genetics and Evolution*, 11(5), 849–854. <https://doi.org/10.1016/j.meegid.2011.01.013>
- Schwarz, E. M., Korhonen, P. K., Campbell, B. E., Young, N. D., Jex, A. R., Jabbar, A., Hall, R. S., Mondal, A., Howe, A. C., Pell, J., Hofmann, A., Boag, P. R., Zhu, X.-Q., Gregory, T., Loukas, A., Williams, B. A., Antoshechkin, I., Brown, C., Sternberg, P. W., & Gasser, R. B. (2013). The genome and developmental transcriptome of the strongylid nematode *Haemonchus contortus*. *Genome Biology*, 14(8), R89. <https://doi.org/10.1186/gb-2013-14-8-r89>
- Seguí, S., Pujol, O., & Vitrià, J. (2015). *Learning to count with deep object features*. Cornell University.
https://openaccess.thecvf.com/content_cvpr_workshops_2015/W03/papers/Segui_Learning_to_Count_2015_CVPR_paper.pdf
- What are convolutional neural networks?* (n.d.). IBM.
<https://www.ibm.com/topics/convolutional-neural-networks>
- Zajac, A., Petersson, K., & Burdett, H. (2014). *How To Do The Modified McMaster Fecal Egg Counting Procedure*. The University of Rhode Island. https://web.uri.edu/wp-content/uploads/sites/241/McMaster-Test_Final3.pdf

Appendix A. Code

The code is referenced from Bandoni (2024).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib
from sklearn.neighbors import KNeighborsClassifier
import sklearn
from sklearn.model_selection import train_test_split
from sklearn import metrics
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
import os
import glob
from glob import glob
import xgboost as xgb
import keras
from keras.datasets import mnist
from keras.layers import Dense
from keras.models import Sequential
import tensorflow as tf
from matplotlib import pyplot as plt
from sklearn.linear_model import LogisticRegression
from random import randint
from sklearn.linear_model import Perceptron
from keras.layers import Dropout
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelBinarizer
from sklearn.model_selection import GridSearchCV
lb=LabelEncoder()
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.ensemble import VotingClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from mpl_toolkits.mplot3d import Axes3D
from tensorflow.keras import layers
import cv2
import tqdm
from sklearn.utils import shuffle
from PIL import Image
from keras.preprocessing.image import ImageDataGenerator
```

```
from keras.layers import Activation, Conv2D, MaxPooling2D, ZeroPadding2D,
GlobalAveragePooling2D, Flatten
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.preprocessing.image import array_to_img, img_to_array, load_img
from ipywidgets import interact
from tensorflow.keras import Model
from tensorflow.keras.optimizers import RMSprop, Adam, SGD
import matplotlib.gridspec as gridspec

from sklearn.metrics import classification_report

!pip install tensorflow-addons

import tensorflow_addons as tfa

#Loading in CSV

from google.colab import drive
drive.mount('/content/drive', force_remount=True)
path = '/content/drive/MyDrive/Data/'
file = path + 'DATA.csv'

eggsdf = pd.read_csv(file)
eggsdf

#Adding image paths to dataframe

img_path = path + 'images/'
Path = []
for x in eggsdf['Image Index']:
    Path.append(img_path + x)
eggsdf['Path'] = Path
eggsdf

#Making a function to show image and essential info

def show_image(img_index):
    img_path = '/content/drive/MyDrive/Data/images/' + img_index
    f=plt.figure(figsize= (20,8))
    s1 = f.add_subplot(1, 2, 1)
    row = eggsdf[eggsdf["Path"] == img_path]
    total = int(row['TOTAL'])
    whole = int(row['Whole Eggs'])
    partial = int(row['Partial Eggs'])
    s1.set_title(f"Slide Image\nImage Index: {img_index}\nTotal Eggs: {total}\nWhole Eggs:
{whole} Partial Eggs: {partial}")
    img = cv2.imread(img_path)
    plt.imshow(img, cmap='gray')
    plt.axis('off')
```

```
plt.show()

show_image('0001_001.JPG')

#Model Time

#Spilting for training, validation, and testing

eggsdf['TOTAL'] = eggsdf['TOTAL'].astype(str)

df_train,df_test = train_test_split(eggsdf,test_size=0.3,stratify=eggsdf["TOTAL"]) # Separate
70% for training and 30% for testing
df_train,df_val = train_test_split(df_train,test_size=0.3) # Of the previous 70% of training
use 30% for training validation
print('Training set:', df_train.shape)
print('Validation in training set:', df_val.shape)
print('Test set:', df_test.shape)

train_datagen = ImageDataGenerator(
    rescale=1./255)#,
    # rotation_range=40,
    # width_shift_range=0.2,
    # height_shift_range=0.2, # Different augments of data
    # shear_range=0.2,
    # zoom_range=0.2,
    # horizontal_flip=True)

# Test set does not need data augmentation
test_datagen = ImageDataGenerator(rescale=1./255)

img_size = 128
batch_size = 4

train_generator = train_datagen.flow_from_dataframe(df_train,
    x_col='Path',
    y_col='TOTAL',
    target_size=(img_size,img_size),
    batch_size = batch_size,
    color_mode='grayscale',
    class_mode = 'categorical')
validation_generator = test_datagen.flow_from_dataframe(df_val,
    x_col='Path',
    y_col='TOTAL',
    target_size=(img_size,img_size),
    batch_size = batch_size,
    color_mode='grayscale',
    class_mode = 'categorical')
```

```
#Model 1

# defining the input layer with the img_size
img_input = layers.Input(shape=(img_size, img_size, 1))

# First convolution extracts 16 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(16, 3, activation='relu')(img_input)
x = layers.MaxPooling2D(2)(x)

# Second convolution extracts 32 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(32, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Third convolution extracts 64 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(64, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Flatten feature map to a 1-dim tensor
x = layers.Flatten()(x)

# Create a fully connected layer with ReLU activation and 512 hidden units
x = layers.Dense(512, activation='relu')(x)

# Add a dropout rate of 0.5
x = layers.Dropout(0.5)(x)

# Create output layer with a single node and sigmoid activation
output = layers.Dense(8, activation='sigmoid')(x)

# Configure and compile the model
m1_model = Model(img_input, output)
m1_model.compile(loss=tfa.losses.SigmoidFocalCrossEntropy(),
                 optimizer=Adam(learning_rate=0.001),
                 metrics=['acc'])

m1_model.summary()

m1_history = m1_model.fit(train_generator,
                        epochs = 1,
                        batch_size = batch_size,
                        validation_data = validation_generator,
                        verbose = 1)

m1_history.history

# Accuracy for training and validation
```

```
m1_acc = m1_history.history['acc']
m1_val_acc = m1_history.history['val_acc']
# Loss for training and validation
m1_loss = m1_history.history['loss']
m1_val_loss = m1_history.history['val_loss']
print(m1_acc)
print(m1_val_acc)

test_datagen = ImageDataGenerator(rescale=1./255)
test_generator = test_datagen.flow_from_dataframe(df_test,
                                                x_col='Path',
                                                y_col=None,
                                                target_size=(img_size,img_size),
                                                batch_size = batch_size,
                                                color_mode='grayscale',
                                                class_mode = None)

m1_pred = m1_model.predict(test_generator)

#Model 2

# defining the input layer with the img_size
img_input = layers.Input(shape=(img_size, img_size, 1))

# First convolution extracts 32 filters that are 5x5
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(32, 5, activation='relu')(img_input)
x = layers.MaxPooling2D(2)(x)

# Second convolution extracts 64 filters that are 4x4
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Conv2D(64, 4, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Third convolution extracts 128 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(128, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Flatten feature map to a 1-dim tensor
x = layers.Flatten()(x)

# Create a fully connected layer with ReLU activation and 512 hidden units
x = layers.Dense(512, activation='relu')(x)

# Add a dropout rate of 0.5
x = layers.Dropout(0.5)(x)

# Create output layer with a single node and sigmoid activation
output = layers.Dense(8, activation='sigmoid')(x)
```

```
# Configure and compile the model
m2_model = Model(img_input, output)
m2_model.compile(loss=tfa.losses.SigmoidFocalCrossEntropy(),
                 optimizer=Adam(learning_rate=0.001),
                 metrics=['acc'])

m2_model.summary()

m2_history = m2_model.fit(train_generator,
                        epochs = 10,
                        batch_size = batch_size,
                        validation_data = validation_generator,
                        verbose = 1)

m2_history.history

# Accuracy for training and validation
m2_acc = m2_history.history['acc']
m2_val_acc = m2_history.history['val_acc']
# Loss for training and validation
m2_loss = m2_history.history['loss']
m2_val_loss = m2_history.history['val_loss']
print(m2_acc)
print(m2_val_acc)

test_datagen = ImageDataGenerator(rescale=1./255)
test_generator = test_datagen.flow_from_dataframe(df_test,
                                                x_col='Path',
                                                y_col=None,
                                                target_size=(img_size,img_size),
                                                batch_size = batch_size,
                                                color_mode='grayscale',
                                                class_mode = None)

m2_pred = m2_model.predict(test_generator)

#Model 3

# defining the input layer with the img_size
img_input = layers.Input(shape=(img_size, img_size, 1))

# First convolution extracts 16 filters that are 6x6
x = layers.Conv2D(16, 6, activation='relu')(img_input)

# Second convolution extracts 32 filters that are 5x5
x = layers.Conv2D(32, 5, activation='relu')(x)

# Third convolution extracts 64 filters that are 4x4
# Convolution is followed by max-pooling layer with a 2x2 window
```

```
x = layers.Convolution2D(64, 4, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Fourth convolution extracts 128 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(128, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Fifth convolution extracts 256 filters that are 2x2
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(256, 2, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Flatten feature map to a 1-dim tensor
x = layers.Flatten()(x)

# Create a fully connected layer with ReLU activation and 512 hidden units
x = layers.Dense(512, activation='relu')(x)

# Add a dropout rate of 0.5
x = layers.Dropout(0.5)(x)

# Create output layer with a single node and sigmoid activation
output = layers.Dense(8, activation='sigmoid')(x)

# Configure and compile the model
m3_model = Model(img_input, output)
m3_model.compile(loss=tfa.losses.SigmoidFocalCrossEntropy(),
                 optimizer=Adam(learning_rate=0.001),
                 metrics=['acc'])

m3_model.summary()

m3_history = m3_model.fit(train_generator,
                        epochs = 50,
                        batch_size = 2,
                        validation_data = validation_generator,
                        verbose = 2)

# Accuracy for training and validation
m3_acc = m3_history.history['acc']
m3_val_acc = m3_history.history['val_acc']
# Loss for training and validation
m3_loss = m3_history.history['loss']
m3_val_loss = m3_history.history['val_loss']
print(m3_acc)
print(m3_val_acc)

#Model 4
```

```
# defining the input layer with the img_size
img_input = layers.Input(shape=(img_size, img_size, 1))

# First convolution extracts 16 filters that are 6x6
x = layers.Conv2D(4, 6, activation='relu')(img_input)
x = layers.MaxPooling2D(2)(x)

# Second convolution extracts 32 filters that are 5x5
x = layers.Conv2D(8, 5, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Third convolution extracts 64 filters that are 4x4
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(16, 4, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Fourth convolution extracts 128 filters that are 3x3
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(32, 3, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Fifth convolution extracts 256 filters that are 2x2
# Convolution is followed by max-pooling layer with a 2x2 window
x = layers.Convolution2D(64, 2, activation='relu')(x)
x = layers.MaxPooling2D(2)(x)

# Flatten feature map to a 1-dim tensor
x = layers.Flatten()(x)

# Create a fully connected layer with ReLU activation and 512 hidden units
x = layers.Dense(128, activation='relu')(x)

# Add a dropout rate of 0.5
x = layers.Dropout(0.5)(x)

# Create output layer with a single node and sigmoid activation
output = layers.Dense(8, activation='sigmoid')(x)

# Configure and compile the model
m4_model = Model(img_input, output)
m4_model.compile(loss=tfa.losses.SigmoidFocalCrossEntropy(),
                 optimizer=Adam(learning_rate=0.001),
                 metrics=['acc'])

m4_model.summary()

m4_history = m4_model.fit(train_generator,
                        epochs = 50,
```

```
        batch_size = 4,
        validation_data = validation_generator,
        verbose = 2)

m4_history.history

# Accuracy for training and validation
m4_acc = m4_history.history['acc']
m4_val_acc = m4_history.history['val_acc']
# Loss for training and validation
m4_loss = m4_history.history['loss']
m4_val_loss = m4_history.history['val_loss']
print(m4_acc)
print(m4_val_acc)

m4_model.save("model")
```